

JavaScript

From Zero to Interview-Ready

A complete preparation guide for the Associate Software Engineer role
(Java Full-Stack) — L1 & L2 technical interviews

*Covers core language, ES6+, async patterns, the DOM,
OOP, closures, common pitfalls, and 100+ interview Q&A.*

Table of Contents

Part I — Foundations

1. Introduction to JavaScript
2. Setting Up & Running JS
3. Variables: var, let, const
4. Data Types & Type Conversion
5. Operators
6. Control Flow (if / switch / loops)

Part II — Core Language

7. Functions (declaration, expression, arrow)
8. Scope, Hoisting & the Temporal Dead Zone
9. Closures
10. The 'this' keyword, call/apply/bind
11. Objects & Object Methods
12. Arrays & Array Methods (map/filter/reduce)
13. Strings & String Methods

Part III — Modern JavaScript (ES6+)

14. Template Literals, Destructuring, Spread/Rest
15. Classes & Inheritance
16. Modules (import/export)
17. Iterators, Generators & Symbols (overview)

Part IV — Asynchronous JavaScript

18. The Event Loop, Stack, Queue
19. Callbacks & Callback Hell
20. Promises
21. async / await
22. Fetch API & AJAX

Part V — Browser & DOM

23. DOM Manipulation
24. Events & Event Delegation
25. Forms, Storage (local/session), JSON

Part VI — Advanced & Practical

26. OOP & Prototypes
27. Error Handling
28. Functional Programming Concepts

29. Common Pitfalls & Best Practices

30. Working with REST APIs (Java backend integration)

Part VII — Interview Preparation

31. L1 Interview Questions (Fundamentals)

32. L2 Interview Questions (Deeper)

33. Coding Problems & Output-Based Questions

34. JavaScript vs Java — Cheat Sheet

35. Final 7-Day Revision Plan

Part I — Foundations

1. Introduction to JavaScript

JavaScript (JS) is a high-level, interpreted, dynamically-typed, single-threaded programming language. It was created by Brendan Eich in 1995 and is governed by the ECMAScript (ECMA-262) specification. While it started as a browser scripting language, today JS runs in many environments — browsers (Chrome's V8, Firefox's SpiderMonkey), servers (Node.js), mobile apps (React Native), and even IoT devices.

Why JavaScript matters in Java Full-Stack roles

In a typical Java full-stack stack (Spring Boot + Angular/React/JSP + REST), the back end is Java but the browser layer is JavaScript. You will write JS to consume REST APIs your Java service exposes, manipulate the DOM, validate forms, and handle asynchronous user interactions. Interviewers test JS to confirm you can be productive on the front-end half of the stack.

Key characteristics

- **Interpreted & JIT compiled:** no separate compile step like Java's `javac`.
- **Dynamically typed:** a variable can hold any type and change types.
- **Prototype-based OOP:** objects inherit directly from other objects (unlike Java's class-based inheritance, though ES6 added `class` syntax sugar).
- **Single-threaded with an event loop:** handles concurrency through asynchronous callbacks rather than threads.
- **First-class functions:** functions are values — can be passed, returned, assigned.

2. Setting Up & Running JavaScript

You have three main ways to run JS while learning:

(a) Browser DevTools Console — Open any browser, press F12, click 'Console' tab. Type any JS and press Enter.

(b) HTML file with `<script>` tag — Create an .html file and embed JS:

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="title">Hello</h1>
    <script>
      console.log("JS is running!");
      document.getElementById("title").textContent = "Hello, JS!";
    </script>
  </body>
</html>
```

(c) Node.js — Install Node.js, save `app.js`, run `node app.js` from terminal. Useful for backend or pure JS practice.

3. Variables: `var`, `let`, `const`

JavaScript has three variable declarations. Choosing the right one is a classic L1 interview question.

Feature	var	let	const
Scope	Function	Block	Block
Re-declarable	Yes	No	No
Re-assignable	Yes	Yes	No
Hoisted	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Recommended	Avoid	When value changes	Default choice

```

var x = 10;           // function-scoped, avoid in modern JS
let y = 20;           // block-scoped, can reassign
const z = 30;         // block-scoped, cannot reassign

// const with objects – the binding is constant, contents are not
const user = { name: "Asha" };
user.name = "Bala";   // OK: mutating property
// user = {};         // ERROR: cannot reassign the binding

```

4. Data Types & Type Conversion

JS has seven primitive types and one non-primitive type. Knowing the difference is essential — primitives are copied by value, objects by reference.

Primitives:

- **Number** — both integers and floats: 42, 3.14
- **String** — "hi", 'hi', `hi`
- **Boolean** — true or false
- **Undefined** — variable declared but not assigned
- **Null** — intentional absence of value
- **BigInt** — for integers beyond $2^{53}-1$: 9007199254740993n
- **Symbol** — unique, immutable identifier

Non-primitive:

- **Object** — includes arrays, functions, dates, regex, plain objects.

typeof operator

```

typeof 42           // "number"
typeof "hi"         // "string"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof null         // "object"    // famous historical bug!
typeof {}           // "object"
typeof []           // "object"
typeof function(){} // "function"

```

Type coercion: == vs ===

== performs type coercion before comparison; === checks both type and value. Always prefer === unless you specifically want coercion.

```

5 == "5"    // true   (coerced)
5 === "5"   // false  (different types)

```

```
null == undefined    // true
null === undefined   // false
0 == false           // true
"" == false          // true   (both coerce to 0)
```

Truthy & Falsy

Falsy values: false, 0, -0, 0n, "", null, undefined, NaN. Everything else is truthy — including "0", "false", [], and {}.

5. Operators

```
// Arithmetic
10 + 3    // 13
10 - 3    // 7
10 * 3    // 30
10 / 3    // 3.333...
10 % 3    // 1   (remainder)
10 ** 3   // 1000 (exponent, ES2016)

// Comparison
5 > 3, 5 < 3, 5 >= 5, 5 !== 6

// Logical (short-circuit)
true && false // false
true || false // true
!true        // false

// Modern operators
null ?? "default" // "default" (nullish coalescing)
undefined ?? 0    // 0
user?.address?.city // optional chaining – safe access
```

6. Control Flow

if / else if / else

```
const score = 75;
if (score >= 90)      grade = "A";
else if (score >= 75) grade = "B";
else if (score >= 60) grade = "C";
else                  grade = "F";

// Ternary
const status = age >= 18 ? "adult" : "minor";
```

switch

```
switch (day) {
  case "Mon":
  case "Tue":
  case "Wed":
  case "Thu":
  case "Fri": console.log("Weekday"); break;
  case "Sat":
  case "Sun": console.log("Weekend"); break;
  default:   console.log("Invalid");
}
```

Loops

```
// classic for
for (let i = 0; i < 5; i++) console.log(i);

// while
let n = 0; while (n < 5) { console.log(n); n++; }

// do-while (runs at least once)
let m = 0; do { console.log(m); m++; } while (m < 3);

// for...of – iterates VALUES of an iterable (array, string, etc.)
for (const ch of "hi") console.log(ch); // h, i

// for...in – iterates KEYS of an object (avoid for arrays!)
for (const key in { a:1, b:2 }) console.log(key); // a, b

// break, continue work as in Java
```

Part II — Core Language

7. Functions

Functions are first-class citizens in JS — they are values and can be stored in variables, passed as arguments, and returned from other functions. This is the foundation of callbacks, higher-order functions, and async patterns.

Three ways to define a function

```
// 1. Function declaration (hoisted fully)
function add(a, b) { return a + b; }

// 2. Function expression (variable hoisted, definition is not)
const sub = function(a, b) { return a - b; };

// 3. Arrow function (ES6) — concise, no own `this`
const mul = (a, b) => a * b;
const sq = x => x * x;
const log = () => console.log("hi");
```

Default, rest & spread parameters

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}

function sum(...nums) {           // rest: collect into array
  return nums.reduce((a, n) => a + n, 0);
}
sum(1, 2, 3, 4);                  // 10

const arr = [1, 2, 3];
console.log(Math.max(...arr));     // spread: 3
```

Arrow vs regular functions — key differences

- Arrow functions do **not** have their own this — they inherit from the enclosing lexical scope.
- No arguments object inside arrows.
- Cannot be used as constructors (new arrow() throws).
- Cannot be used as object methods if you need this to refer to the object.

8. Scope, Hoisting & the Temporal Dead Zone

Scope defines where a variable is accessible. JS has three: global, function, and (since ES6) block scope.

```
let g = "global";
function outer() {
  let f = "function-scoped";
  if (true) {
    let b = "block-scoped";
    var v = "function-scoped (var ignores blocks)";
  }
  console.log(v);    // OK — var leaks out of the block
  // console.log(b); // ReferenceError
}
```


Hoisting: JS moves declarations to the top of their scope at compile time. `var` declarations are hoisted and initialized to `undefined`. `let` and `const` are hoisted but *not* initialized — accessing them before declaration throws a `ReferenceError`. The period from start of scope to the declaration line is called the **Temporal Dead Zone (TDZ)**.

```
console.log(a);    // undefined (var is hoisted & initialized)
var a = 5;

console.log(b);    // ReferenceError – TDZ
let b = 5;

// Function DECLARATIONS are hoisted fully
hello();           // works
function hello() { console.log("hi"); }

// Function EXPRESSIONS are not
bye();             // TypeError: bye is not a function
var bye = function() { console.log("bye"); };
```

9. Closures

A **closure** is a function bundled with references to the variables in its surrounding (lexical) scope. Even after the outer function returns, the inner function still has access to those variables. Closures power data privacy, memoization, and module patterns. **Almost every L2 interview asks about closures.**

```
function makeCounter() {
  let count = 0;           // private variable
  return function() {
    count++;
    return count;
  };
}

const counter = makeCounter();
counter();    // 1
counter();    // 2
counter();    // 3
// `count` is inaccessible from outside – encapsulation!
```

Classic closure pitfall with `var` in a loop

```
// PROBLEM: prints 3, 3, 3 – all callbacks share the SAME `i`
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}

// FIX: use `let` – each iteration gets its own block-scoped i
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);    // prints 0, 1, 2
}
```

10. The 'this' Keyword, call/apply/bind

this is dynamic — it refers to the execution context, determined by **how** a function is called, not where it's defined. This is the biggest mental shift for Java developers, where this is the current instance.

```
// 1. Global call -> globalThis (window in browser), undefined in strict mode
function show() { console.log(this); }
show();

// 2. Method call -> the object before the dot
const user = {
  name: "Asha",
  greet() { console.log(`Hi, ${this.name}`); }
};
user.greet();           // Hi, Asha

// 3. Constructor call -> the new instance
function Car(model) { this.model = model; }
const c = new Car("i20"); // c.model === "i20"

// 4. Arrow function -> inherits this from enclosing scope (lexical)
const obj = {
  name: "X",
  m1: function() { return () => this.name; }, // inner arrow -> obj
  m2: () => this.name                       // outer arrow -> globalThis
};
```

call, apply, bind — explicit binding

```
function intro(greeting, punct) {
  return `${greeting}, I am ${this.name}${punct}`;
}
const p = { name: "Asha" };

intro.call(p, "Hello", "!"); // call: args one-by-one
intro.apply(p, ["Hi", "."]); // apply: args as array
const bound = intro.bind(p, "Hey"); // bind: returns new function
bound("?"); // "Hey, I am Asha?"
```

11. Objects & Object Methods

```
// Object literal
const book = {
  title: "JS Deep Dive",
  pages: 320,
  author: { name: "K", country: "IN" },
  read() { return `Reading ${this.title}`; } // method shorthand
};

// Access
book.title; // dot notation
book["title"]; // bracket — needed for dynamic keys
const key = "pages"; book[key]; // 320

// Add / delete
book.year = 2024;
delete book.pages;

// Iterate keys
for (const k in book) console.log(k, book[k]);
```

```
// Useful static methods
Object.keys(book);           // array of keys
Object.values(book);         // array of values
Object.entries(book);        // array of [k, v] pairs
Object.freeze(book);         // make immutable
const copy = { ...book };    // shallow copy with spread
const merged = Object.assign({}, a, b); // merge
```

Pass by value vs pass by reference

Primitives are passed by value (a copy). Objects (including arrays) are passed by *reference value* — the variable holds a reference, and mutating the object inside a function affects the caller's object.

```
function changePrim(x) { x = 100; }
function changeObj(o) { o.name = "Z"; }

let n = 10;          changePrim(n); console.log(n);          // 10
let u = {name:"A"}; changeObj(u);   console.log(u.name);     // "Z"
```

12. Arrays & Higher-Order Methods

```
const nums = [10, 20, 30, 40];

nums.length;           // 4
nums[0];               // 10
nums.push(50);          // add to end
nums.pop();             // remove from end
nums.unshift(5);        // add to start
nums.shift();           // remove from start
nums.indexOf(30);       // 2
nums.includes(20);       // true
nums.slice(1, 3);        // [20, 30] – non-mutating
nums.splice(1, 2);       // [20, 30] – MUTATES, removes 2 from idx 1
nums.concat([60,70]);
nums.reverse();         // mutates
nums.sort((a,b)=>a-b); // numeric sort (default sort is alphabetical!)
```

The big three: map, filter, reduce

```
const nums = [1, 2, 3, 4, 5];

// map – transform each element, returns new array
const squared = nums.map(n => n * n);           // [1,4,9,16,25]

// filter – keep elements where predicate is true
const evens = nums.filter(n => n % 2 === 0);    // [2, 4]

// reduce – fold array into single value
const sum = nums.reduce((acc, n) => acc + n, 0); // 15

// chain them
const totalOfEvenSquares = nums
  .filter(n => n % 2 === 0)
  .map(n => n * n)
  .reduce((a, n) => a + n, 0);                  // 4 + 16 = 20

// forEach – runs a function, returns undefined (use for side effects)
nums.forEach(n => console.log(n));
```

```
// find / findIndex / some / every
nums.find(n => n > 3);           // 4
nums.findIndex(n => n > 3);      // 3
nums.some(n => n > 4);           // true
nums.every(n => n > 0);          // true

// flat / flatMap
[[1,2],[3,[4]]].flat();         // [1,2,3,[4]]
[[1,2],[3,[4]]].flat(Infinity); // [1,2,3,4]
```

13. Strings & String Methods

```
const s = "Hello, JS World";

s.length;                // 15
s.toUpperCase();          // "HELLO, JS WORLD"
s.toLowerCase();
s.indexOf("JS");          // 7
s.includes("World");      // true
s.startsWith("Hello");    // true
s.endsWith("World");      // true
s.slice(7, 9);            // "JS"
s.substring(7, 9);        // "JS"
s.replace("JS", "JavaScript");
s.replaceAll(",", ";");
s.split(" ");             // ["Hello,", "JS", "World"]
s.trim();                 // remove leading/trailing whitespace
s.padStart(20, "*");
s.repeat(3);

// Template literals (backticks)
const name = "Asha";
`Hello, ${name}! You have ${5 + 2} messages.`
```

Part III — Modern JavaScript (ES6+)

14. Template Literals, Destructuring, Spread/Rest

Template literals use backticks and support `${...}` interpolation and multi-line strings.

```
const name = "Asha", age = 27;
const msg = `Hello, ${name}!
You are ${age} years old.
In 10 years you'll be ${age + 10}.`;
```

Destructuring unpacks values from arrays or objects.

```
// Array destructuring
const [a, b, c] = [1, 2, 3];
const [first, ...rest] = [10, 20, 30, 40]; // first=10, rest=[20,30,40]

// Swap variables
let x = 1, y = 2; [x, y] = [y, x];

// Object destructuring
const user = { name: "Asha", age: 27, city: "Chennai" };
const { name, age } = user;

// Rename + default
const { name: userName, country = "IN" } = user;

// Nested destructuring
const { address: { city } = {} } = user;

// In function parameters (very common)
function printUser({ name, age }) {
  console.log(`${name}, ${age}`);
}
printUser(user);
```

Spread & Rest use the same `...` syntax.

```
// Spread – expand into individual elements
const a = [1, 2], b = [3, 4];
const merged = [...a, ...b]; // [1,2,3,4]
const objMerged = { ...user, age: 30 }; // override age

// Rest – collect remaining into array/object
function sum(...nums) { return nums.reduce((a,n)=>a+n,0); }
const { name, ...others } = user; // others has all keys except name
```

15. Classes & Inheritance

ES6 introduced class syntax — syntactic sugar over JS's prototype-based inheritance. The mental model is similar to Java but the underlying mechanism is different.

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return `${this.name} makes a sound.`;
  }
  static create(name) { // static method
```

```

    return new Animal(name);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);           // call parent constructor
    this.breed = breed;
  }
  speak() {               // override
    return `${this.name} barks.`;
  }
  describe() {
    return `${super.speak()} It's a ${this.breed}.`;
  }
}

const d = new Dog("Rex", "Labrador");
d.speak();    // "Rex barks."
d instanceof Dog;    // true
d instanceof Animal; // true

```

Getters, setters & private fields

```

class Account {
  #balance = 0;           // # = truly private (ES2022)
  get balance() { return this.#balance; }
  set balance(v) {
    if (v < 0) throw new Error("Negative");
    this.#balance = v;
  }
}

const a = new Account();
a.balance = 500;
console.log(a.balance);    // 500

```

16. Modules (import / export)

Modern JS code is organized into modules — each file is its own scope. Use export to expose things and import to use them. In HTML, load with `<script type='module' src='app.js'>`.

```

// math.js — named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export function sub(a, b) { return a - b; }

// logger.js — default export
export default function log(msg) {
  console.log(`${new Date().toISOString()} ${msg}`);
}

// app.js — importing
import log from "./logger.js";           // default
import { PI, add } from "./math.js";     // named
import * as math from "./math.js";       // namespace
import { add as addNumbers } from "./math.js"; // rename

log(`PI is ${PI}, 2+3 = ${add(2,3)}`);

```

17. Iterators, Generators & Symbols (Overview)

Brief because L1/L2 interviews rarely deep-dive here, but you should recognize the syntax:

```
// Generator – pausable function with yield
function* idGen() {
  let id = 1;
  while (true) yield id++;
}
const gen = idGen();
gen.next().value; // 1
gen.next().value; // 2

// Symbol – unique value, useful for object keys to avoid collisions
const SECRET = Symbol("secret");
const obj = { [SECRET]: 42 };
```

Part IV — Asynchronous JavaScript

18. The Event Loop, Call Stack & Queues

JS is **single-threaded** — only one piece of code runs at any moment. Yet it handles network calls, timers, and user clicks without blocking. The secret is the **event loop**.

Components

- **Call Stack** — where synchronous code is pushed and popped.
- **Web APIs / Node APIs** — handle async work (timers, fetch, DOM events) outside the JS engine.
- **Callback Queue (Macrotask Queue)** — completed `setTimeout`, `setInterval`, I/O, UI events wait here.
- **Microtask Queue** — `Promise .then/catch/finally` and `queueMicrotask` callbacks wait here. Drained completely *before* the next macrotask.
- **Event Loop** — checks the stack; when empty, drains all microtasks, then takes one macrotask.

Classic ordering question

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// Output: 1, 4, 3, 2
// Why? Sync first (1, 4). Microtask (Promise) before macrotask (setTimeout).
```

19. Callbacks & Callback Hell

A **callback** is a function passed to another function, to be called later (often after async work completes). It's the oldest async pattern in JS.

```
function fetchUser(id, callback) {
  setTimeout(() => callback({ id, name: "Asha" }), 1000);
}
fetchUser(1, (user) => console.log(user));
```

Callback hell happens when async operations depend on each other:

```
login(user, (err, token) => {
  if (err) return handle(err);
  fetchProfile(token, (err, profile) => {
    if (err) return handle(err);
    fetchOrders(profile.id, (err, orders) => {
      if (err) return handle(err);
      render(orders);    // pyramid of doom
    });
  });
});
```

Promises and `async/await` solve this by flattening the structure.

20. Promises

A **Promise** is an object representing the eventual result of an async operation. It has three states: **pending**, **fulfilled**, or **rejected**. Once settled, the state never changes.


```
// Creating a promise
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    const ok = Math.random() > 0.5;
    ok ? resolve("Success!") : reject(new Error("Failed"));
  }, 500);
});

// Consuming
p.then(value => console.log(value))
  .catch(err => console.error(err))
  .finally(() => console.log("done"));

// Chaining – each then returns a new promise
fetchUser(1)
  .then(user => fetchOrders(user.id))
  .then(orders => render(orders))
  .catch(err => console.error(err));
```

Static methods

```
Promise.all([p1, p2, p3])
  // resolves when ALL resolve, rejects on first rejection
  .then([r1, r2, r3]) => ...);

Promise.allSettled([p1, p2, p3])
  // waits for all, returns array of {status, value/reason}

Promise.race([p1, p2])
  // settles with the FIRST one to settle

Promise.any([p1, p2])
  // resolves with the first fulfilled, rejects only if ALL reject
```

21. async / await

async/await (ES2017) is syntactic sugar over promises that lets you write async code that *looks* synchronous. An async function always returns a Promise. await pauses execution until the promise settles.

```
async function loadDashboard(userId) {
  try {
    const user    = await fetchUser(userId);
    const orders  = await fetchOrders(user.id);
    const profile = await fetchProfile(user.id);
    return { user, orders, profile };
  } catch (err) {
    console.error("loadDashboard failed:", err);
    throw err;
  }
}

// Run in parallel when calls are independent – much faster
async function loadFast(userId) {
  const [user, orders] = await Promise.all([
    fetchUser(userId),
    fetchOrders(userId),
  ]);
  return { user, orders };
}
```

22. Fetch API & AJAX

The Fetch API is the modern way to make HTTP requests from JS. It returns a Promise. (Older code uses XMLHttpRequest.) This is exactly how your front end will talk to your Java/Spring Boot REST endpoints.

```
// GET
async function getUsers() {
  const res = await fetch("https://api.example.com/users");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}

// POST
async function createUser(user) {
  const res = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": "Bearer " + token,
    },
    body: JSON.stringify(user),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}

// PUT, DELETE, PATCH – same shape, different method
await fetch(`/users/${id}`, { method: "DELETE" });
```

Part V — Browser & DOM

23. DOM Manipulation

The **Document Object Model** is a tree representation of an HTML document. JS reads and modifies the page through this tree.

```
// Selection
document.getElementById("title");
document.querySelector(".card");           // first match
document.querySelectorAll(".card");        // NodeList of all
document.getElementsByClassName("x");      // live HTMLCollection
document.getElementsByTagName("p");

// Content
el.textContent = "Hello";                  // text only, safer
el.innerHTML   = "<b>Hi</b>";               // parses HTML – XSS risk!
el.value;                                       // for form inputs

// Attributes
el.getAttribute("href");
el.setAttribute("data-id", "42");
el.dataset.id;                               // "42" – via data-* attrs

// Classes
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("on");
el.classList.contains("active");

// Styles (inline)
el.style.color = "red";
el.style.backgroundColor = "#eee";

// Create / append / remove
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);
li.remove();

// Traversal
el.parentElement; el.children;
el.firstElementChild; el.nextElementSibling;
```

24. Events & Event Delegation

```
// addEventListener – the modern way
btn.addEventListener("click", (e) => {
  console.log("Clicked", e.target);
  e.preventDefault(); // stop form submit / link navigation
  e.stopPropagation(); // stop bubbling up
});

// Remove a listener – must pass the SAME function reference
function handler(e) { /* ... */ }
btn.addEventListener("click", handler);
btn.removeEventListener("click", handler);
```

Event delegation

Instead of attaching a listener to every child, attach one to the parent and use `event.target` to find which child fired. Efficient and handles dynamically added children automatically.

```
document.getElementById("todo-list").addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    e.target.classList.toggle("done");
  }
  if (e.target.matches(".delete-btn")) {
    e.target.closest("li").remove();
  }
});
```

Event flow: capturing → target → bubbling

By default, listeners fire during the **bubbling** phase (target up to root). Pass `{ capture: true }` as a third arg to listen during the **capturing** phase (root down to target).

25. Forms, Storage & JSON

```
// Form submit
document.querySelector("#loginForm").addEventListener("submit", (e) => {
  e.preventDefault();
  const fd = new FormData(e.target);
  const data = Object.fromEntries(fd.entries());
  // data = { email: "...", password: "..." }
  login(data);
});

// localStorage – persists until cleared
localStorage.setItem("token", "abc123");
const t = localStorage.getItem("token");
localStorage.removeItem("token");
localStorage.clear();

// sessionStorage – cleared when tab closes – same API

// Store an object – must serialize
localStorage.setItem("user", JSON.stringify({name:"Asha", age:27}));
const u = JSON.parse(localStorage.getItem("user"));

// JSON
JSON.stringify({a: 1, b: [2,3]}); // '{"a":1,"b":[2,3]}'
JSON.parse('{"a":1}');           // {a: 1}
```

Part VI — Advanced & Practical

26. Prototypal Inheritance

Behind class syntax, JS uses prototype chains. Every object has an internal link to another object called its **prototype**. When you access a property, JS walks up the chain until it finds it or reaches null.

```
const animal = {
  eat() { console.log(`${this.name} eats`); }
};

const dog = Object.create(animal); // dog's prototype is animal
dog.name = "Rex";
dog.eat(); // "Rex eats"

// Chain: dog -> animal -> Object.prototype -> null
Object.getPrototypeOf(dog) === animal; // true

// Constructor function (pre-ES6 style)
function Person(name) { this.name = name; }
Person.prototype.greet = function() {
  return `Hi, ${this.name}`;
};
const p = new Person("Asha");
p.greet(); // greet is found on Person.prototype
```

27. Error Handling

```
try {
  const data = JSON.parse(input); // may throw SyntaxError
  doWork(data);
} catch (err) {
  if (err instanceof SyntaxError) {
    console.error("Bad JSON:", err.message);
  } else {
    console.error("Other error:", err);
  }
} finally {
  cleanup(); // always runs
}

// Throwing custom errors
class ValidationError extends Error {
  constructor(msg, field) {
    super(msg);
    this.name = "ValidationError";
    this.field = field;
  }
}

if (!email.includes("@")) {
  throw new ValidationError("Invalid email", "email");
}

// async errors – use try/catch with await, or .catch() on the promise
try { await fetchData(); } catch (e) { handle(e); }
```

28. Functional Programming Concepts

- **Pure functions** — given same input always return same output, no side effects.
- **Immutability** — don't mutate; produce new values ([...arr, x] instead of arr.push(x)).
- **Higher-order functions** — accept or return functions (map/filter/reduce).
- **Composition** — build complex behavior by combining small functions.
- **Currying** — transform a function of many args into a chain of one-arg functions.

```
// Currying example
const multiply = a => b => a * b;
const double   = multiply(2);
const triple   = multiply(3);
double(5);     // 10
triple(5);     // 15

// Composition
const compose = (f, g) => x => f(g(x));
const addExclaim = s => s + "!";
const upper      = s => s.toUpperCase();
const shout      = compose(addExclaim, upper);
shout("hi");     // "HI!"
```

29. Common Pitfalls & Best Practices

- Always use === instead of ==.
- Never use var in new code — use const by default, let if you need to reassign.
- Beware floating-point math: 0.1 + 0.2 === 0.30000000000000004.
- typeof null returns "object" — a legacy bug.
- Arrays' default sort is alphabetical ([10,2,1].sort() → [1,10,2]). Use comparator: arr.sort((a,b) => a - b).
- for...in iterates keys (for objects), for...of iterates values (for arrays/iterables).
- NaN !== NaN. Use Number.isNaN(x).
- Don't mutate function arguments; treat objects/arrays as immutable when possible.
- Be careful with this inside callbacks — use arrow functions or .bind().
- Don't use innerHTML with user input — XSS risk. Use.textContent or sanitize first.
- Promises that aren't awaited or chained silently swallow errors. Always handle rejections.

30. Working with REST APIs (Java Backend Integration)

In your full-stack role you'll typically have a Spring Boot REST controller like:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping public List<User> all() { ... }
    @GetMapping("/{id}") public User one(@PathVariable Long id) { ... }
    @PostMapping public User create(@RequestBody User u) { ... }
}
```

Your JS front end consumes it like this:

```
const API = "http://localhost:8080/api/users";

async function getAll() {
    const res = await fetch(API);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
}
```

```
    return res.json();          // Spring serializes to JSON automatically
  }

  async function create(user) {
    const res = await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(user),
      credentials: "include",    // send cookies for session auth
    });
    if (!res.ok) throw new Error(await res.text());
    return res.json();
  }

  // Render
  getAll()
    .then(users => {
      const ul = document.getElementById("users");
      users.forEach(u => {
        const li = document.createElement("li");
        li.textContent = `${u.id} - ${u.name}`;
        ul.appendChild(li);
      });
    })
    .catch(err => console.error(err));
```

CORS reminder

When your JS is served from one origin and your Java API from another, the browser blocks requests unless your Spring config enables CORS (@CrossOrigin or a CorsConfiguration bean). This is a frequent L1/L2 follow-up question.

Part VII — Interview Preparation

31. L1 Interview Questions (Fundamentals)

L1 (round 1) typically tests fundamentals, definitions, and short coding. Aim for crisp 30–60-second answers.

Q1. What is JavaScript? Is it the same as Java?

JavaScript is a high-level, dynamically-typed, interpreted programming language created by Brendan Eich (1995) and standardized as ECMAScript. Despite the similar name, it is unrelated to Java — different syntax, different type system, different runtime. Java is a statically-typed, compiled, class-based language; JS is dynamic and prototype-based.

Q2. Difference between var, let, and const?

var is function-scoped, hoisted with value undefined, and re-declarable. let is block-scoped, hoisted but in the TDZ (so accessing before declaration throws), and re-assignable. const is identical to let except it cannot be re-assigned after initialization — the binding is constant, though object contents can still mutate.

Q3. == vs === ?

== performs type coercion before comparing. === checks both type and value with no coercion. Prefer === to avoid surprises like `0 == false` → true.

Q4. List the data types in JavaScript.

Seven primitives: Number, String, Boolean, Undefined, Null, BigInt, Symbol. One non-primitive: Object (which includes arrays and functions).

Q5. What is hoisting?

JavaScript moves variable and function declarations to the top of their scope at compile time. var declarations are hoisted and initialized to undefined; let/const are hoisted but stay in the Temporal Dead Zone until their declaration line. Function declarations are hoisted with their body, so you can call them before they appear in code.

Q6. What is the difference between null and undefined?

undefined means a variable was declared but never assigned (or a function argument was not supplied). null is an explicit assignment by the developer to indicate 'no value'. `typeof undefined` is 'undefined'; `typeof null` is 'object' (historical bug).

Q7. What is NaN? How do you check for it?

NaN means 'Not a Number' but its type is still 'number'. It's the only JS value that is not equal to itself, so `NaN === NaN` is false. Use `Number.isNaN(x)` to check reliably.

Q8. What is a callback function?

A function passed as an argument to another function so it can be called later, typically after an asynchronous operation completes. Example: `setTimeout(() => console.log('done'), 1000)`.

Q9. What is a Promise? What are its states?

A Promise is an object representing the eventual completion or failure of an async operation. States: pending (initial), fulfilled (success — with a value), rejected (failure — with a reason). Once settled, state

never changes.

Q10. What is the difference between map, filter, and reduce?

All are array methods that take a callback. map transforms each element and returns a new array of the same length. filter keeps only elements for which the callback returns truthy, returning a new array of equal-or-shorter length. reduce collapses the array into a single value using an accumulator.

Q11. What is the DOM?

The Document Object Model — a live, tree-structured representation of an HTML document that JS can read and modify through APIs like getElementById, querySelector, and addEventListener.

Q12. What does 'use strict' do?

Enables strict mode — a more restrictive variant of JS. Catches common bugs: throws errors on undeclared variables, makes this undefined in standalone functions, forbids duplicate parameter names, etc. ES6 modules use it by default.

Q13. What is an IIFE?

Immediately Invoked Function Expression — a function defined and called at once: `(function(){ /* ... */ })()`. Used historically to create private scope before let/const and modules existed.

Q14. How do you copy an array?

`arr.slice()`, `[...arr]`, `Array.from(arr)` — all produce shallow copies. For deep copy of nested data use `structuredClone(arr)` (modern) or `JSON.parse(JSON.stringify(arr))` for JSON-safe data.

Q15. What is the difference between forEach and map?

forEach executes a callback for each element but returns undefined — used for side effects. map returns a new array containing the callback's return values — used for transformation.

Q16. What does JSON stand for? How do you convert objects and JSON?

JavaScript Object Notation — a text format for data interchange. `JSON.stringify(obj)` converts an object to a JSON string; `JSON.parse(str)` converts a JSON string back to an object.

Q17. What is event bubbling?

When an event fires on an element, it 'bubbles up' through its ancestors, firing the same event type on each one, up to the document root. The opposite phase is capturing (root → target). Stop bubbling with `event.stopPropagation()`.

Q18. What is the difference between innerHTML and textContent?

innerHTML returns/sets the HTML markup inside an element — it parses HTML, which is an XSS risk when used with untrusted input. textContent returns/sets only the text — safer and faster.

Q19. What is a template literal?

A string written with backticks that supports interpolation via `${expr}` and multi-line strings without escape characters.

Q20. What are arrow functions and how do they differ from regular functions?

Arrow functions (ES6) are a concise function syntax: `(a,b) => a+b`. They do not have their own this, arguments, or super — they inherit them from the enclosing lexical scope. They also cannot be used as constructors.

32. L2 Interview Questions (Deeper)

L2 (round 2 / hiring manager) probes deeper understanding, edge cases, and design intuition. Expect follow-ups; explain your reasoning aloud.

Q1. Explain closures with a real-world use case.

A closure is a function that retains access to its lexical environment even after the outer function returns. Practical uses: (a) data privacy — private counters or bank-balance modules; (b) function factories like `makeMultiplier(n)`; (c) memoization caches; (d) maintaining state in event handlers without polluting global scope.

Q2. Explain the event loop with microtasks and macrotasks.

JS is single-threaded. The call stack runs sync code. Async APIs (timers, fetch, DOM events) hand off work to the browser/Node runtime. When that work completes, callbacks are queued — Promise callbacks go to the microtask queue, `setTimeout/setInterval/I/O` to the macrotask queue. The event loop checks: if the stack is empty, drain ALL microtasks, then take ONE macrotask, repeat. That's why `Promise.then` runs before `setTimeout(0)`.

Q3. How does prototypal inheritance differ from classical inheritance?

In classical (Java) inheritance, classes are templates from which instances are created; relationships are fixed at compile time. In prototypal inheritance, objects inherit directly from other objects via a prototype link, looked up at runtime. ES6 class syntax in JS is sugar over the same prototype mechanism — there are no real classes under the hood.

Q4. What is the difference between `call`, `apply`, and `bind`?

All three set the value of `this` for a function. `call` invokes immediately, passing args one-by-one. `apply` invokes immediately, passing args as an array. `bind` does NOT invoke — it returns a new function with `this` and (optionally) initial arguments permanently bound.

Q5. What is debouncing and throttling? When would you use each?

Both limit how often a function runs. Debouncing delays execution until input stops for N ms (good for search-as-you-type, resize end). Throttling ensures execution at most once per N ms (good for scroll handlers, mouse-move). Debounce = wait for quiet; Throttle = enforce rate.

Q6. What is the difference between shallow copy and deep copy?

Shallow copy duplicates the top-level structure; nested objects are still shared by reference. Achieved via `spread`, `Object.assign`, `slice`. Deep copy duplicates everything recursively. Achieved via `structuredClone()`, or `JSON.parse(JSON.stringify(x))` for JSON-safe data, or a recursive helper.

Q7. Why does `typeof null` return `'object'`?

A bug in the first JavaScript implementation: values had type tags in their low bits; the tag for objects was 0, and null was represented as the null pointer (all zeros), so `typeof null` became `'object'`. Fixing it would break the web, so it stays.

Q8. Explain `async/await` error handling versus `.catch()`.

With `async/await`, wrap awaits in `try/catch` — exceptions thrown anywhere in the awaited chain are caught synchronously-looking. With `.then().catch()`, you attach a `.catch` handler to the promise chain. `async/await` is usually cleaner for sequential logic; `.catch()` is fine for short chains. Either way, never leave a rejection unhandled.

Q9. What is event delegation and why is it useful?

Attaching a single event listener to a common ancestor instead of many child elements, then using `event.target` to identify which child triggered the event. Benefits: fewer listeners (lower memory), automatically handles dynamically added/removed children, simpler teardown.

Q10. Promise.all vs Promise.allSettled vs Promise.race vs Promise.any?

`all`: resolves with all values when all fulfill; rejects on first rejection. `allSettled`: waits for all, returns `{status, value/reason}` array — never rejects. `race`: settles with whichever first settles (fulfilled or rejected). `any`: resolves on first fulfillment; rejects with `AggregateError` only if all reject.

Q11. What is currying? Why use it?

Currying transforms a function of `N` arguments into a chain of `N` functions each taking one argument. `f(a,b,c)` becomes `f(a)(b)(c)`. Useful for partial application, building reusable specialized functions, and composing in functional pipelines.

Q12. What is the difference between localStorage, sessionStorage, and cookies?

`localStorage`: ~5–10MB, persists until cleared, never sent with HTTP requests, same-origin. `sessionStorage`: same API but cleared when the tab closes. Cookies: ~4KB, sent automatically with every request to the origin — used for server-side session/auth; configurable expiry, `HttpOnly`, `Secure`, `SameSite` flags.

Q13. What is CORS? How would you enable it for a Spring Boot backend?

Cross-Origin Resource Sharing — a browser security mechanism that blocks JS from reading responses across origins unless the server opts in with `Access-Control-Allow-Origin` (and related) headers. In Spring Boot, you can add `@CrossOrigin` on the controller, or register a global `CorsConfigurationSource` bean, or configure it via `WebMvcConfigurer`.

Q14. What are higher-order functions? Give an example.

Functions that take other functions as arguments or return them. Examples: `Array.prototype.map/filter/reduce`, `setTimeout`, `addEventListener`. Custom: `function once(fn){ let done=false,val; return (...a)=>{ if(!done){val=fn(...a);done=true;} return val; } }`

Q15. Explain the difference between synchronous and asynchronous code.

Synchronous code executes line-by-line and blocks the thread until each step completes. Asynchronous code initiates work that finishes later, freeing the thread to do other things; the result is delivered via callbacks, promises, or events. Since JS is single-threaded, `async` is how it stays responsive under I/O.

Q16. What is memoization?

Caching the results of expensive function calls keyed by their arguments, so repeated calls with the same inputs return instantly. Easily implemented with a closure over a `Map`: `const memo = fn => { const c = new Map(); return (...a) => { const k = JSON.stringify(a); if (!c.has(k)) c.set(k, fn(...a)); return c.get(k); } }`

Q17. What is the difference between Object.freeze() and const?

`const` prevents reassignment of the variable binding — but the object it points to is still mutable. `Object.freeze(obj)` makes the object's properties read-only (shallowly — nested objects can still mutate).

Q18. What are the new features of ES6 (you must know)?

let/const, arrow functions, template literals, default parameters, rest/spread, destructuring, classes, modules (import/export), Promises, Map/Set, Symbols, for...of, generators, computed property names, shorthand object syntax.

Q19. How does JS handle memory management?

Automatic garbage collection — typically a mark-and-sweep algorithm. The engine starts from roots (globals, current call stack) and marks reachable objects; unreachable ones are reclaimed. Common leak sources: forgotten timers/intervals, detached DOM nodes still referenced, accidental globals, closures holding large data, and event listeners not removed.

Q20. How would you optimize a slow web page (front-end)?

Reduce bundle size (code-splitting, tree-shaking, minification). Defer non-critical JS (async/defer). Lazy-load images and routes. Debounce expensive handlers. Cache (HTTP cache, service worker, in-memory). Avoid layout thrash — batch DOM reads/writes. Use CDN. Compress (gzip/Brotli). Measure with Lighthouse/DevTools Performance.

Q21. What is the difference between Map/Set and Object/Array?

Object keys are strings or Symbols only; Map keys can be any value, including objects, and Map preserves insertion order with predictable iteration. Map also has size and is generally faster for frequent add/remove. Set stores unique values with $O(1)$ has/add — better than Array.includes for membership checks.

Q22. Explain the difference between var hoisting and let/const hoisting.

All three are hoisted to the top of their scope. var is also initialized to undefined at hoist time, so accessing it before its declaration yields undefined. let and const remain uninitialized in the Temporal Dead Zone until the declaration line; accessing them earlier throws a ReferenceError.

Q23. What is the spread operator vs rest parameter?

Same ... syntax, opposite roles. Spread expands an iterable/object into individual elements/properties: [...arr], {...obj}, fn(...args). Rest collects multiple elements into one array/object: function f(...args), const [a, ...rest] = arr, const { a, ...others } = obj.

Q24. How does JavaScript handle concurrency without threads?

Through the event loop and non-blocking I/O. While the engine itself is single-threaded, the runtime (browser/Node) provides background facilities for timers, network, and file I/O. When work completes, callbacks are queued and run on the main thread once the stack is clear. CPU-heavy work can be offloaded to Web Workers (browser) or worker_threads (Node).

33. Coding Problems & Output-Based Questions

Practice writing these from scratch — they appear frequently in associate-level interviews.

P1. Reverse a string

```
function reverse(s) {
  return s.split("").reverse().join("");
}

// without built-ins
function reverse2(s) {
  let r = "";
  for (let i = s.length - 1; i >= 0; i--) r += s[i];
  return r;
}
```

P2. Check palindrome

```
function isPalindrome(s) {
  s = s.toLowerCase().replace(/[^\a-z0-9]/g, "");
  return s === s.split("").reverse().join("");
}
```

P3. FizzBuzz

```
for (let i = 1; i <= 100; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}
```

P4. Find the largest number in an array

```
const arr = [3, 8, 2, 15, 6];
Math.max(...arr); // 15
arr.reduce((m, x) => x > m ? x : m, -Infinity);
```

P5. Remove duplicates from an array

```
const arr = [1, 2, 2, 3, 4, 4];
[...new Set(arr)]; // [1,2,3,4]
arr.filter((v, i, a) => a.indexOf(v) === i); // same
```

P6. Count word/character frequency

```
function freq(str) {
  const m = {};
  for (const ch of str) m[ch] = (m[ch] || 0) + 1;
  return m;
}
freq("javascript"); // {j:1,a:2,v:1,s:1,c:1,r:1,i:1,p:1,t:1}
```

P7. Fibonacci series

```
function fib(n) {
  const a = [0, 1];
  for (let i = 2; i < n; i++) a.push(a[i-1] + a[i-2]);
  return a.slice(0, n);
}
fib(8); // [0,1,1,2,3,5,8,13]
```

```
// recursive (not great for large n)
const fibR = n => n < 2 ? n : fibR(n-1) + fibR(n-2);
```

P8. Factorial

```
const fact = n => n <= 1 ? 1 : n * fact(n - 1);
fact(5);    // 120
```

P9. Flatten a nested array

```
const arr = [1, [2, [3, [4]]]];
arr.flat(Infinity);           // [1,2,3,4]

// manual recursion
function flatten(a) {
  return a.reduce(
    (acc, v) => acc.concat(Array.isArray(v) ? flatten(v) : v),
    []
  );
}
```

P10. Debounce

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
const onSearch = debounce(q => api.search(q), 300);
input.addEventListener("input", e => onSearch(e.target.value));
```

P11. Throttle

```
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

Output-based — predict and explain

```
// Q1
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Output: 1, 4, 3, 2    (microtask before macrotask)

// Q2
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Output: 3, 3, 3    (var shared; loop completes before timers)
```

```
// Q3
console.log(typeof null);
console.log(typeof undefined);
console.log([] + []);
console.log([] + {});
console.log({} + []);
// Output: "object", "undefined", "", "[object Object]", "[object Object]"

// Q4
const a = { x: 1 };
const b = a;
b.x = 99;
console.log(a.x);    // 99 – same reference

// Q5
console.log(0.1 + 0.2 === 0.3);    // false (floating point)
console.log(0.1 + 0.2);            // 0.30000000000000004
```

34. JavaScript vs Java — Quick Cheat Sheet

Concept	Java	JavaScript
Typing	Static, compiled	Dynamic, interpreted (JIT)
Variables	<code>int x = 5; String s;</code>	<code>let x = 5; const s = '';</code>
Equality	<code>==</code> (refs), <code>.equals()</code>	<code>===</code> (strict), <code>==</code> (coerces)
Class system	Class-based	Prototype-based (class is sugar)
Inheritance	<code>extends</code> , <code>implements</code>	<code>extends</code> , prototype chain
Method ref / λ	Lambda <code>(a,b) -> a+b</code>	Arrow <code>(a,b) => a+b</code>
Collections	List, Set, Map	Array, Set, Map
Streams / pipe	<code>stream().map().filter()</code>	<code>arr.map().filter()</code>
Null safety	<code>Optional<T></code>	<code>?.</code> and <code>??</code> operators
Async model	Threads, <code>CompletableFuture</code>	Single thread + event loop
Async syntax	<code>CompletableFuture</code> , <code>ExecutorService</code>	Promise, <code>async/await</code>
JSON	Jackson/Gson	<code>JSON.parse</code> / <code>JSON.stringify</code>
HTTP client	<code>RestTemplate</code> , <code>WebClient</code>	<code>fetch</code> / <code>axios</code>
Build tool	Maven, Gradle	npm, yarn, vite, webpack
Runtime	JVM	V8 / SpiderMonkey / Node.js

35. Final 7-Day Revision Plan

Use this if your interview is one week away. Adjust to your pace.

Day 1 — Foundations

Variables, types, operators, control flow. Solve 5 small problems (reverse string, palindrome, FizzBuzz, max in array, factorial).

Day 2 — Functions & scope

Declarations vs expressions vs arrows; default/rest/spread; scope, hoisting, TDZ. Practice writing closures.

Day 3 — Objects, arrays, ES6

Object literals, destructuring, spread/rest, classes. Master `map/filter/reduce` by writing them from scratch.

Day 4 — Async

Event loop diagram on paper. Callbacks → Promises → `async/await`. Write a small project that fetches data and renders it.

Day 5 — DOM & events

Build a small to-do app or live search using DOM APIs, event delegation, and `fetch`. Use `localStorage`.

Day 6 — Tricky concepts

this, call/apply/bind, prototypes, closures pitfalls, output-prediction problems. Re-read section 32 (L2 questions).

Day 7 — Mock interview

30-min self-mock: 10 L1 questions, 5 L2 questions, 2 coding problems. Review section 34 (Java↔JS cheat sheet) the night before.

Tips for interview day

- Think aloud — interviewers grade reasoning more than the final answer.
- When stuck, narrow the problem with a tiny example.
- If unsure of an answer, say so and reason from first principles.
- Mention edge cases unprompted (empty array, null input, large N).
- After coding, suggest one optimization or test case before they ask.
- Be honest about what you've used vs only read about — they will dig.

Good luck! You've prepared. Trust the work, breathe, and have a real conversation with the interviewer — that's what hiring managers actually remember.